# INTEGRATED PROJECT SYNOPSIS

**ON**

# ATHENA

**AI Code Provenance Tracker**

submitted to

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**

for

**INTEGRATED PROJECT (22CS038)**


**Submitted By -**
Arshdeep Singh(2310991708)
Abhishek Singla(2310991677)
**Semester:** 6th
**Session:** 2023-27

**Submitted To –**
Dr. Himanshu Gupta

# INDEX


| Sr. No. | Sub Title | Page No. |
|---|---|---|
| **1.** | Problem Statement | 1 |
| **2.** | Objectives & Key Learnings | 2 |
| **3.** | Functional & Non-Functional Requirements | 3-4 |
| **4.** | High-Level & Low-Level Design | 5-13 |
| **5.** | Advantages & Disadvantages | `14 |
| **6.** | Conclusion & Future Improvements | 15-16 |
| **7.** | References | 17 |

# 1. Problem Statement

Developers increasingly commit AI-generated code into production codebases without adequate review. Code produced by AI coding assistants such as GitHub Copilot, Cursor, and ChatGPT frequently contains subtle security vulnerabilities, hallucinated API calls that reference non-existent methods, deprecated patterns, and logic errors that appear syntactically correct but are semantically flawed. According to the Harness State of Software Delivery Report 2025, 67% of developers spend more time debugging AI-generated code and 68% spend more time resolving security vulnerabilities introduced by such code.

Additionally, security analysis of third-party or open-source repositories currently requires manually cloning, installing multiple tools, and interpreting fragmented output — a slow and inconsistent process with no unified interface or severity framework.

## Current limitations include:

- No existing static analysis tool specifically identifies AI-generated code sections for elevated scrutiny — all code is treated equally regardless of how it was produced, leaving security teams blind to the highest-risk unreviewed sections.
- Existing pre-commit tools (ESLint, Husky) flag only syntax and style violations; they do not assess security risk in relation to code origin or flag hallucinated API usage.
- Developers have no automated mechanism to detect hallucinated imports and non-existent method calls that LLMs produce with high confidence, which pass compilation but fail silently at runtime.
- Security auditing of external repositories requires local setup and produces fragmented output across multiple tools with no cross-referenced severity report.
- Teams lack temporal visibility — there is no tracking of whether a codebase's AI involvement ratio is increasing over time and whether risk is growing as a result.

Athena addresses these challenges through a dual-surface architecture: a CLI tool published as an npm package (@arsh342/athena) that operates as a pre-commit security gate with diff-aware analysis on the developer's local machine, and a full-stack web platform where users paste any GitHub repository URL and receive live streaming security analysis rendered in a browser-based sandbox terminal. Both surfaces share the same core engine (@athena/core) — AST parsing, 11-signal heuristic scoring, static analysis via ESLint and Semgrep, native secret detection, and hallucinated API detection.

**Title of Project:**

Athena — AI Code Provenance Tracker: A dual-surface developer security platform consisting of (1) a CLI tool (@arsh342/athena) that detects likely AI-generated code, runs targeted static analysis, and blocks insecure commits via a pre-commit hook, and (2) a full-stack web platform where users submit GitHub repository URLs and receive live streaming security analysis in a browser-based sandbox terminal backed by PostgreSQL persistence.

# 2. Objectives & Key Learnings

## 2.1 Objectives

- Design an AI code provenance detection engine that parses JS/TS/JSX/TSX files using the TypeScript Compiler API (ts.createSourceFile), extracts function, class, method, and arrow function units, and scores each unit using 11 configurable weighted heuristic signals.
- Develop a targeted security analysis pipeline that runs ESLint (programmatic API with eslint-plugin-security), Semgrep (optional subprocess), native secret detection (regex + Shannon entropy), and hallucinated API detection specifically on flagged units, cross-referencing findings with AI likelihood scores.
- Implement diff-aware analysis for the pre-commit hook — parsing git diff to extract changed line ranges and scoring only affected units, producing a risk delta report per commit rather than a full-repo scan.
- Build a repository fingerprinting system that computes a baseline from the first full scan and uses deviation scoring on subsequent scans to enable context-aware anomaly detection calibrated to each project's own coding style.
- Build a CLI tool with animated terminal output (Ora, Chalk) published to npm as @arsh342/athena that installs as a git pre-commit hook via athena init and blocks commits containing CRITICAL or HIGH severity findings.
- Implement a full-stack web platform where users submit GitHub URLs, the backend clones repositories into isolated sandboxes, spawns the core engine as a child process, streams live output to a browser xterm.js terminal via WebSockets, and saves results to PostgreSQL via Prisma ORM.
- Deliver a JWT-authenticated dashboard with scan history, AI percentage metrics, risk density, confidence distribution, and temporal trend analysis across scans.

## 2.2 Key Learnings

- AST traversal and programmatic code analysis using the TypeScript Compiler API — extracting structured units without a full compilation pass using ts.createSourceFile.
- Heuristic design for stylometric classification — building 11 independently-weighted signals covering naming patterns, comment behavior, structural uniformity, and LLM-specific patterns (emoji in comments, boilerplate templates).
- Shannon entropy as a signal — applying information theory to both identifier naming (low entropy = generic names = AI signal) and secret detection (high entropy = potential API key).
- Integration of multiple static analysis engines into a unified pipeline with graceful degradation — ESLint always runs, Semgrep degrades cleanly if not installed.
- Real-time output streaming from Node.js child processes to browser clients via WebSocket — handling data events, backpressure, connection lifecycle, and per-session isolation.
- Browser terminal emulation using xterm.js v5 with fit and web-links addons for live ANSI-rendered output in a React component.
- Secure child process sandboxing — isolated temp directories, 100MB repo size limits, 120-second scan timeouts, and guaranteed cleanup on completion, error, or disconnect.
- Full-stack web development: React + Vite frontend, Node.js + Express backend, PostgreSQL with Prisma ORM, JWT authentication with bcrypt, WebSocket protocol alongside REST API.
- Monorepo architecture with npm workspaces — three packages sharing a common core engine with zero coupling between CLI and web layers.

# 3. Functional & Non-Functional Requirements

## 3.1 Functional Requirements

| ID | Feature | Description |
|---|---|---|
| **FR1** | User Authentication | Secure registration and login with JWT tokens, bcrypt password hashing (12 salt rounds), and protected API routes |
| **FR2** | AST Parsing | TypeScript Compiler API parses JS/TS/JSX/TSX files into function, class, method, and arrow function units with full structural metadata |
| **FR3** | 11-Signal Heuristic Scoring | Each code unit scored 0-100 for AI likelihood using 11 configurable weighted signals; units above threshold (default: 65) flagged |
| **FR4** | Diff-Aware Analysis | Pre-commit mode parses git diff to extract changed line ranges and scores only affected units, producing a risk delta report per commit |
| **FR5** | Repo Fingerprinting | First scan computes a baseline stored in .athena/baseline.json; subsequent scans compute deviation from project's own coding style |
| **FR6** | Security Analysis | ESLint programmatic API + Semgrep subprocess (optional) + native secret detector + hallucination detector run on flagged units only |
| **FR7** | Severity Classification | Findings cross-referenced with AI score using rule taxonomy: CRITICAL / HIGH / MEDIUM / LOW per finding type × AI score band |
| **FR8** | Suppression System | athena suppress <id> writes to .athena/suppressions.json; engine skips suppressed findings on subsequent scans |
| **FR9** | CLI Pre-commit Hook | athena init installs hook; exit code 1 blocks commit on CRITICAL/HIGH findings; exit code 0 allows commit |
| **FR10** | Animated Terminal Output | Ora spinners + Chalk color coding produce structured animated terminal report with severity icons and signal breakdown |
| **FR11** | GitHub Repo Scanning | Web platform accepts GitHub URL, clones to isolated temp directory (/tmp/athena-scanId/), validates size, runs core engine |
| **FR12** | Live WebSocket Streaming | Backend pipes child process stdout/stderr to browser via WebSocket; xterm.js renders live ANSI output in browser terminal |
| **FR13** | Scan History Dashboard | Completed scan results including repo, timestamp, AI percentage, risk density, and findings saved to PostgreSQL per user account |
| **FR14** | Temporal Trend Analysis | CLI athena report --trend and web dashboard display AI score direction (increasing/decreasing/stable) and change percentage across scans |
| **FR15** | Benchmark Runner | athena benchmark runs core engine against labeled corpus of AI-generated and human-written code, reporting Precision, Recall, and F1 score |

## 3.2 Non-Functional Requirements

| ID | Attribute | Requirement |
|---|---|---|
| **NF1** | Performance | CLI scans 50-file repository in under 10 seconds; single file under 1 second; web clone + scan in under 30 seconds |
| **NF2** | Security | CLI fully offline — zero external API calls, no telemetry; web sandbox isolated per scan with 100MB size limit and 120-second timeout |
| **NF3** | Portability | CLI compatible with Node.js 18+ on macOS, Linux, and Windows (WSL); no platform-specific configuration required |
| **NF4** | Scalability | Web backend handles max 3 concurrent scans; rate limited to 10 scans per hour per user via express-rate-limit |
| **NF5** | Usability | Single-command CLI install: npm install -g @arsh342/athena && athena init; web platform requires only a GitHub URL to scan |
| **NF6** | Maintainability | Monorepo with shared @athena/core engine — single source of truth; TypeScript strict mode; modular signal architecture for ablation studies |
| **NF7** | Reliability | Graceful degradation: missing Semgrep → ESLint-only mode; parse failure → skip file, continue scan; WS disconnect → scan continues, result available via GET |
| **NF8** | Data Integrity | PostgreSQL with Prisma ORM; scan results stored as JSONB; indexed on userId and createdAt for dashboard queries |

# 4. High-Level & Low-Level Design

## 4.1 High-Level Architecture

The system follows a three-layer architecture with a shared core engine. Two entry points (CLI and web) consume the same @athena/core package, ensuring identical analysis logic across both surfaces.

**Presentation Layer:**

- CLI surface: Commander.js commands, animated terminal output (Chalk + Ora), git pre-commit hook integration.
- Web surface: React + Vite SPA with pages for Landing, Dashboard, Scan, Report, Login, and Register. Key components: Terminal (xterm.js wrapper), ScoreGauge (AI percentage visualization), ReportView (severity cards + code blocks), ProtectedRoute, Navbar.

**Business Logic Layer (@athena/core):**

- Engine Orchestrator — coordinates AST Parser, Heuristic Scorer, Security Analyzer, Diff Analyzer, Repo Fingerprint, Hallucination Detector, Suppression Manager, and Report Generator.
- Web Backend — Express server with REST API (/api/auth/*, /api/scans/*, /api/reports/*), WebSocket server for live streaming, Sandbox Manager for isolated repo cloning, JWT auth middleware, and rate limiting middleware.

**Data Layer:**

PostgreSQL (web): Users, Scans, Suppressions tables managed via Prisma ORM. JSON file store (CLI): .athena/history/ for scan history, .athena/baseline.json for repo fingerprint, .athena/suppressions.json for suppressed findings.

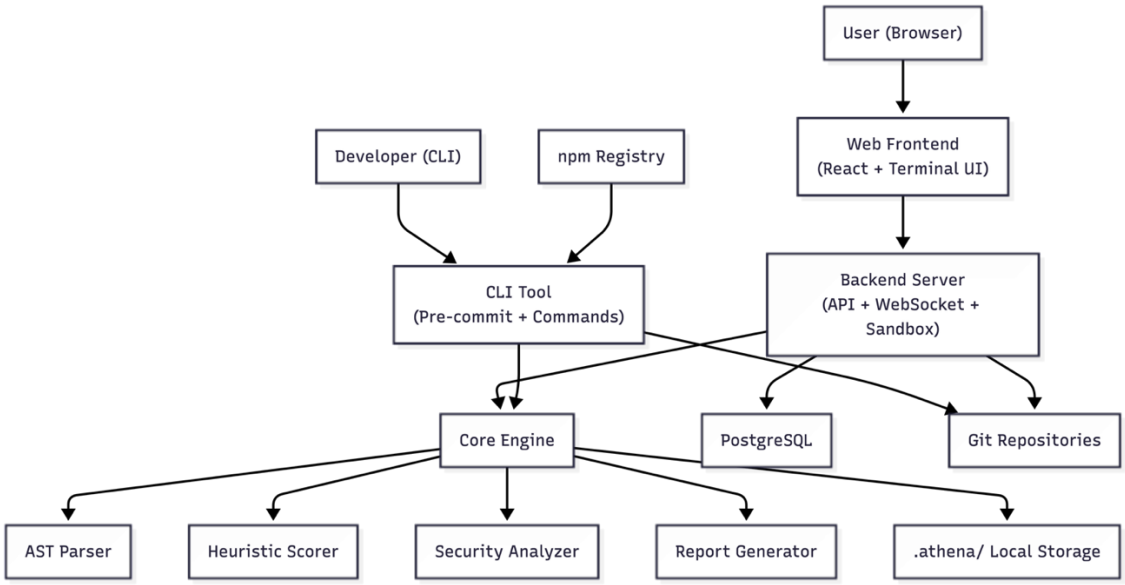


Fig. 4.1.1 Athena System Architecture Diagram shows the interaction between CLI, web interface, backend, core engine, and external systems.

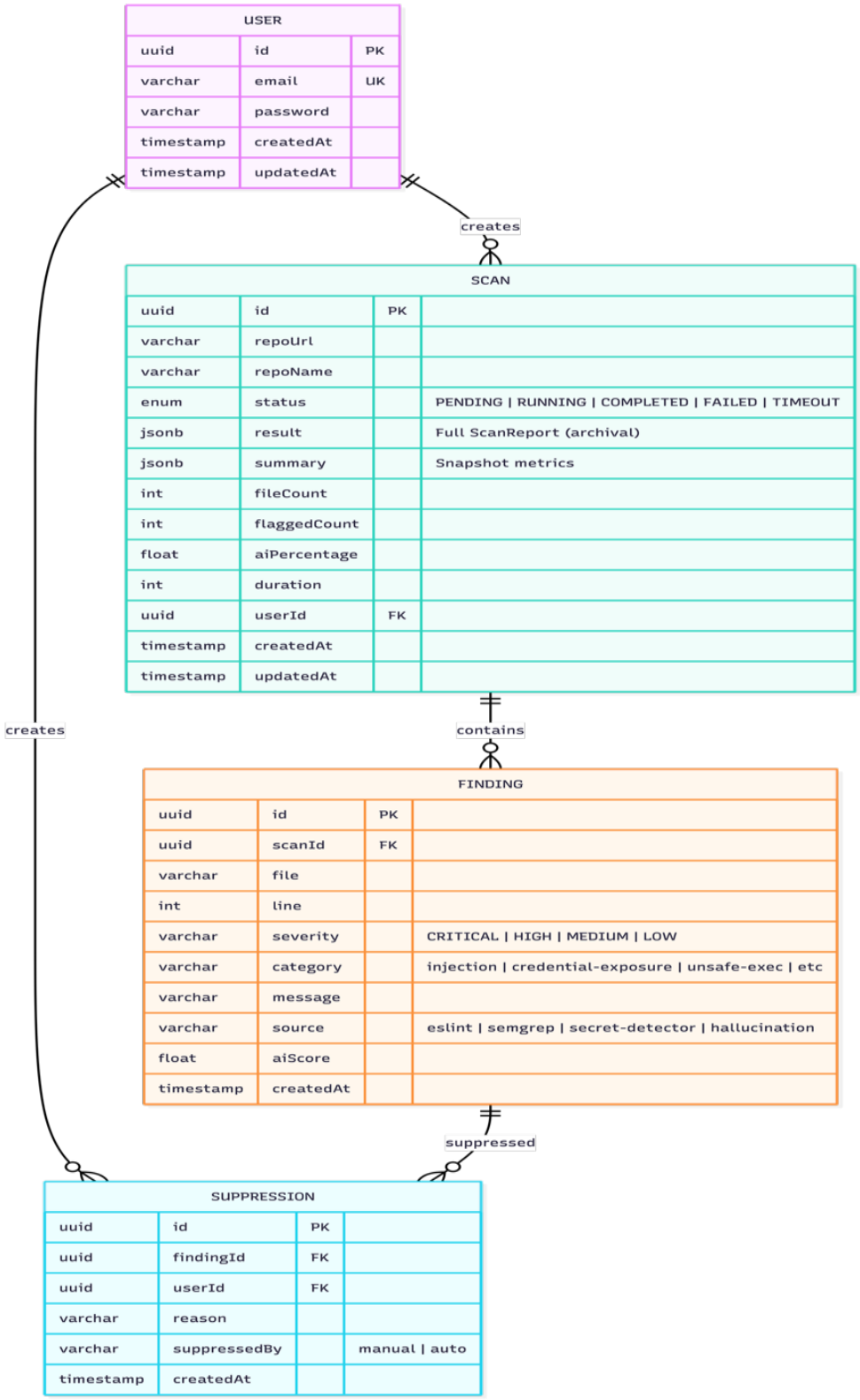


Fig. 4.1.2 Athena ER Diagram shows the data model with USER, SCAN, FINDING, and SUPPRESSION entities and their relationships.

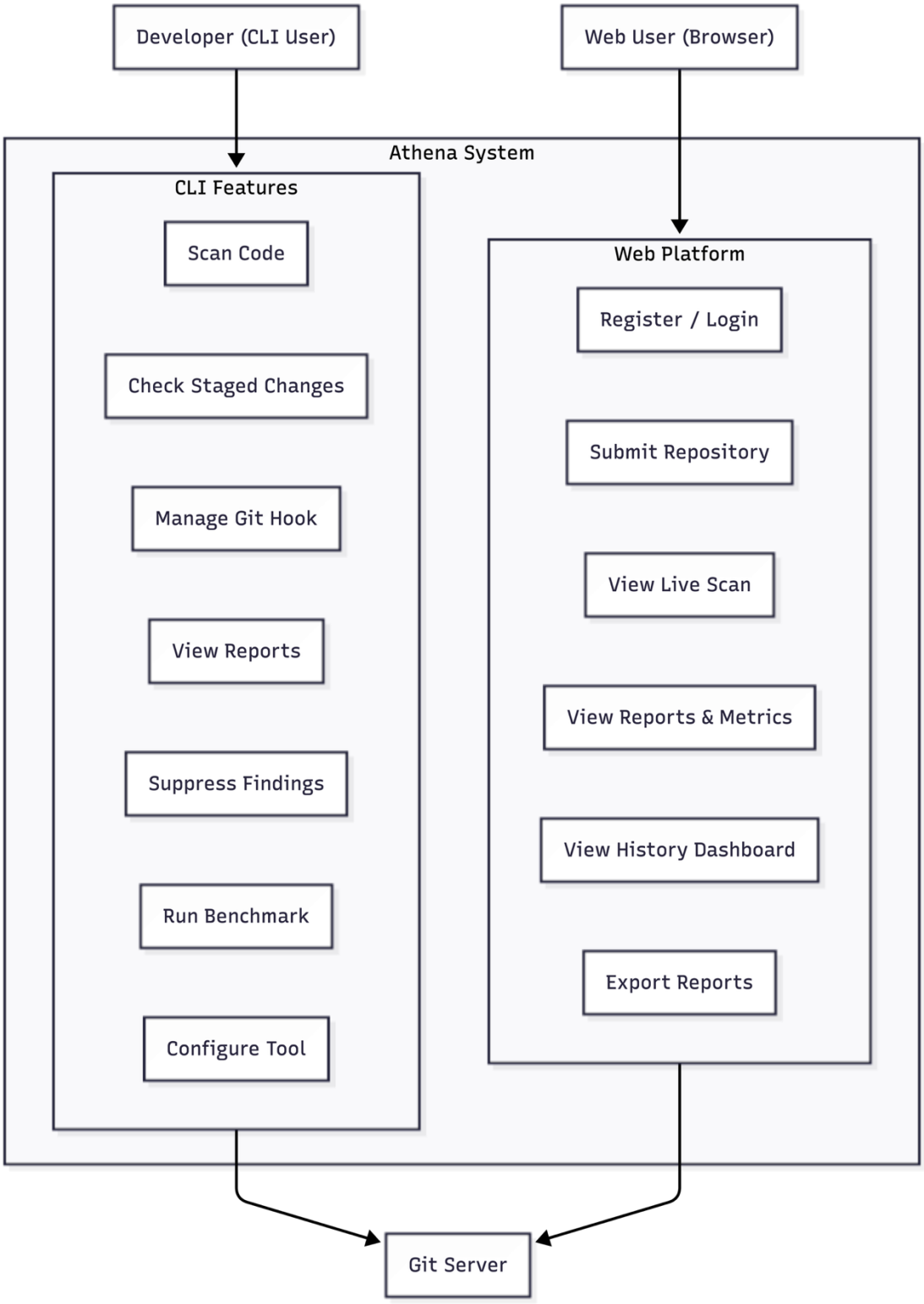


Fig. 4.1.3 Athena Use Case Diagram shows how developers and web users interact with Athena's CLI and web features.

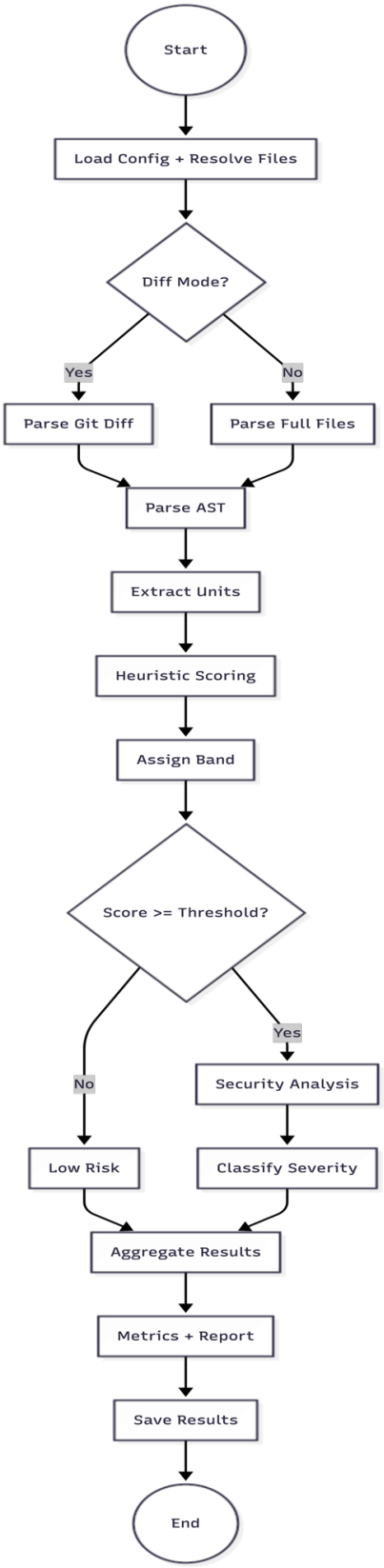


Fig. 4.1.4 Athena Activity Diagram shows the overall workflow from file processing to analysis and report generation.

## 4.2 Low-Level Design & Algorithm

### AST Parser:

ts.createSourceFile() parses each file without a full type-checking program (faster, no type resolution needed). Recursive ts.forEachChild() visitor extracts FunctionDeclaration, ArrowFunction, MethodDeclaration, and ClassDeclaration nodes. Each node yields a CodeUnit with: id (hash), name, kind, filePath, startLine, endLine, raw code, lines of code, comment lines, comment ratio, all identifiers, nesting depth, parameters, JSDoc presence, and cyclomatic complexity estimate. Units under 3 LOC are skipped.

### 11-Signal Heuristic Scorer:

Each code unit is evaluated against 11 independent signals. Raw scores are summed (maximum 110) and normalized to 0-100. Signals and weights:

| # | Signal | Weight | Method |
|---|---|---|---|
| **1** | Generic Variable Names | 15 | Count identifiers matching generic wordlist (data, result, temp, item, value) ÷ total identifiers |
| **2** | Excessive Comment Ratio | 12 | commentRatio > 0.4 = max score; linear scale below threshold |
| **3** | Obvious-Code Comments | 10 | Comment immediately preceding single assignment, return statement, or console.log |
| **4** | Empty Catch Blocks | 10 | AST: catch clause with empty body or console.error only, no recovery logic |
| **5** | Unnecessary Null Checks | 8 | Optional chaining on required parameters; redundant typeof checks on typed variables |
| **6** | Formatting Uniformity | 8 | Indentation standard deviation across unit equals zero — inhuman consistency |
| **7** | Universal JSDoc Coverage | 7 | Every function and method in unit has preceding comment block |
| **8** | Low Naming Entropy | 10 | Shannon entropy of identifier character set below project baseline threshold |
| **9** | Boilerplate Patterns | 10 | Template matching: CRUD function signatures, Express route handlers, React boilerplate |
| **10** | Helper Function Ordering | 10 | Helpers consistently defined before use; sequential call pattern typical of LLM output |
| **11** | Emoji in Comments | 10 | Presence of emoji characters (✅🔥📝⚡🚀💡) in comments — strong AI-generation signal |

Confidence bands: LOW (0-40, likely human), UNCERTAIN (40-70, ambiguous), HIGH (70-100, likely AI). Each scored unit includes explainability output: top 3 signals by percentage contribution. Weights are fully configurable via .athena.config.json enabling ablation studies. Repo fingerprinting adjusts scores relative to each project's own baseline — converting generic detection into context-aware anomaly detection.

**Severity Classification Matrix:**

| Finding Type | AI Score ≥ 65 | AI Score < 65 |
|---|---|---|
| Hardcoded secret / credential | 🔴 CRITICAL | 🟠 HIGH |
| Known vulnerability (injection, XSS, eval) | 🟠 HIGH | 🟡 MEDIUM |
| Insecure pattern (weak regex, timing attack) | 🟡 MEDIUM | 🔵 LOW |
| Hallucinated API call / invalid import | 🟠 HIGH | 🟡 MEDIUM |
| Code quality (empty catch, unused var) | 🟡 MEDIUM | 🔵 LOW |

**WebSocket Streaming Architecture:**

Backend spawns core engine using Node.js spawn() with the repository directory as argument. process.stdout and process.stderr data events are piped directly to ws.send() per chunk. The frontend xterm.js instance writes received chunks via term.write(). Connection closes on process exit. Each scan session uses a UUID-keyed room to prevent cross-session data leakage. If the WebSocket disconnects mid-scan, the scan continues and results are saved to PostgreSQL for retrieval via GET /api/scans/:id.

**Sandbox Manager:**

Each web scan creates a UUID-named directory under /tmp/athena-scanId/. Repository is cloned with --depth 1 (shallow clone). A 100MB size guard is enforced via du check after clone. A 120-second absolute timeout kills the child process and deletes the temp directory regardless of state. Cleanup also runs on WebSocket disconnect and server restart. Maximum 3 concurrent scans enforced at the service level.

**REST API Endpoint Design:**

The backend follows RESTful conventions. Authentication: POST /api/auth/register and POST /api/auth/login return JWT tokens. Scans: POST /api/scans accepts { repoUrl } and returns { scanId, wsUrl }; GET /api/scans returns the user's scan history; GET /api/scans/:id returns a full scan report. WebSocket protocol: client connects to ws://host/ws/scan/:scanId and receives JSON messages of types: log (streaming output), progress (stage + percent), result (full ScanReport), and error.

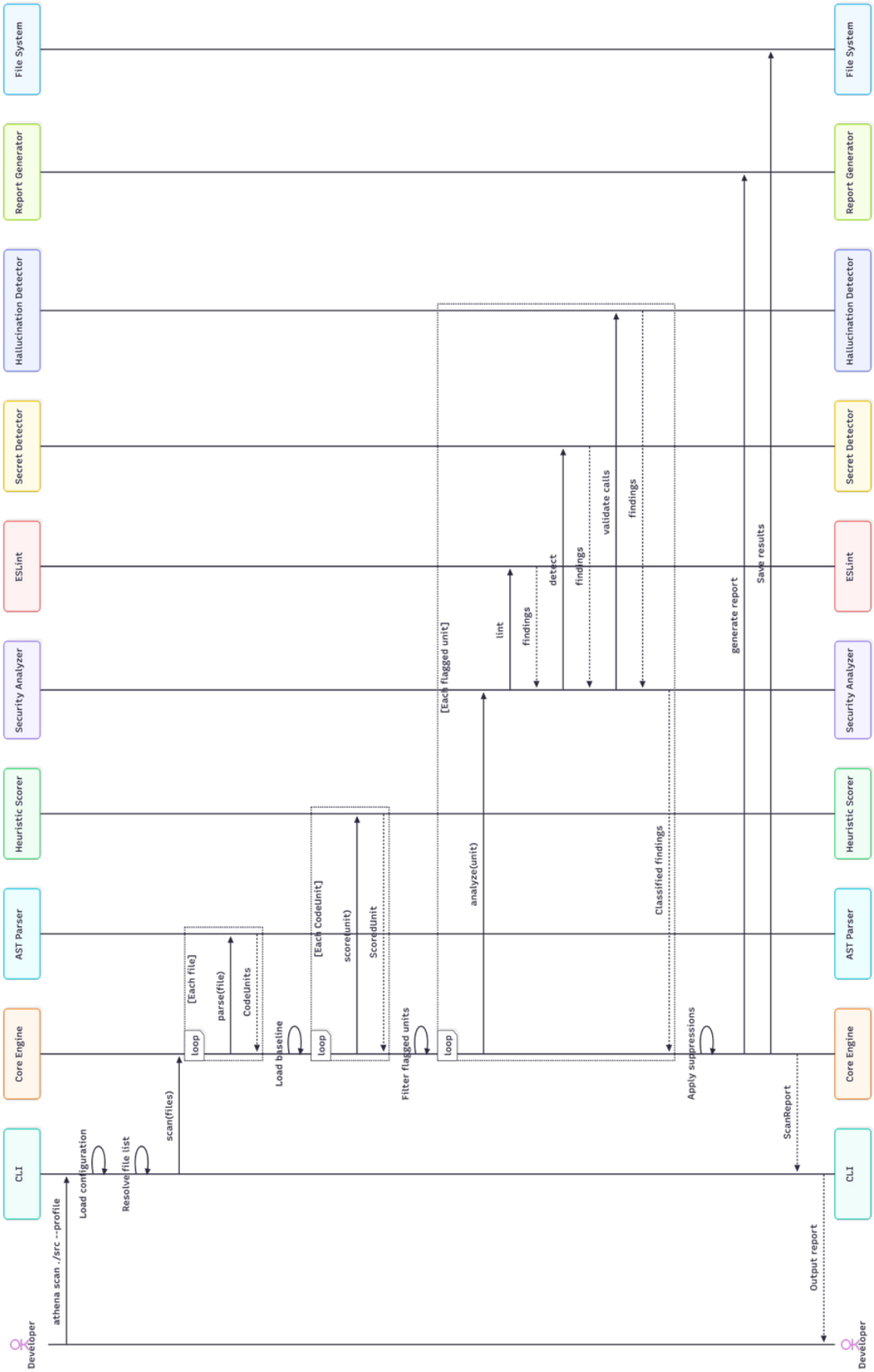


Fig. 4.2.1 Athena CLI Scan Flow shows how code is scanned via CLI through parsing, analysis, and report generation.

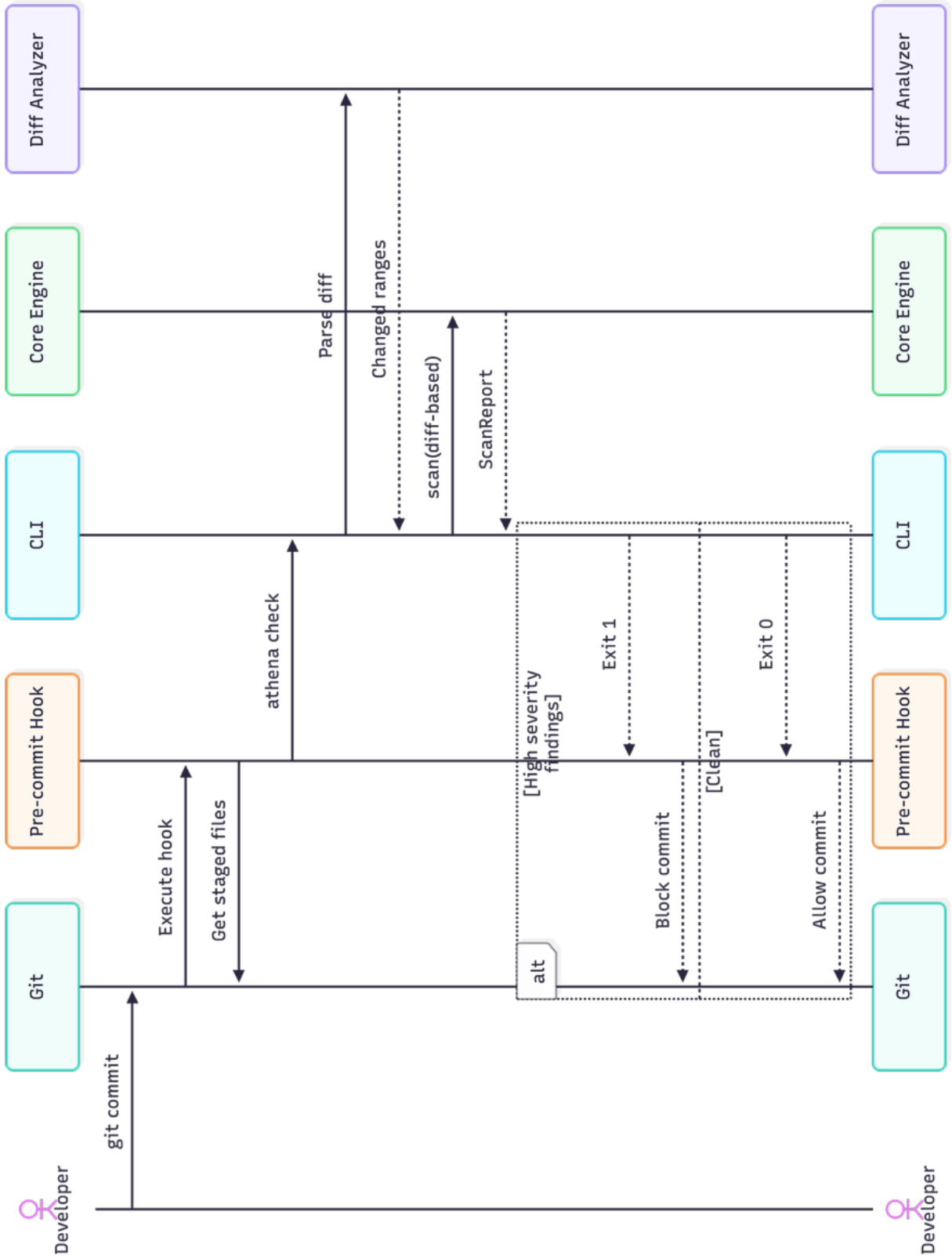


Fig. 4.2.2 Athena Pre-commit Hook Flow shows how staged code is checked before commit and blocked if critical issues are found.

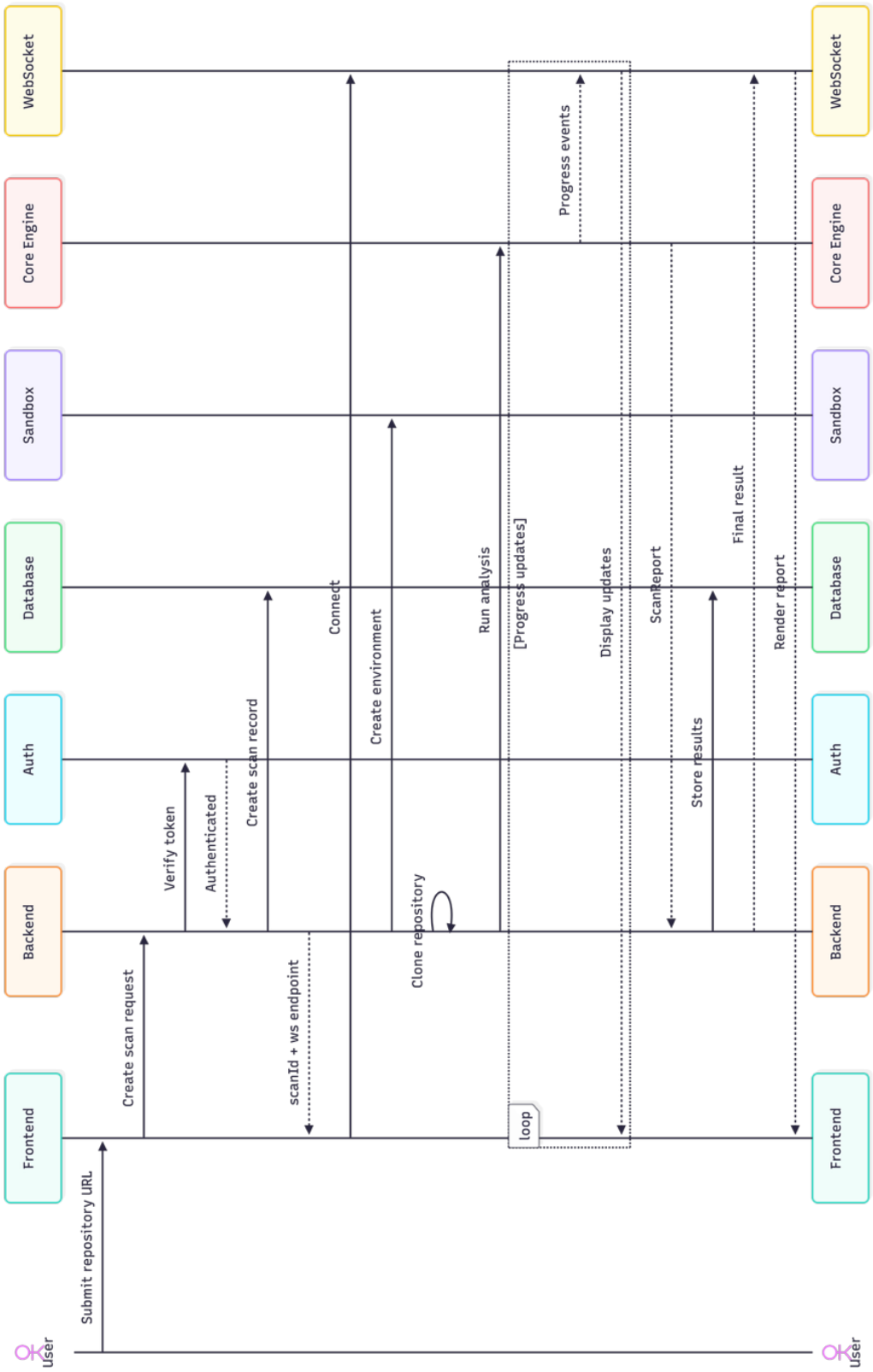


Fig. 4.2.3 Athena Web Scan Flow shows how a repository is scanned via the web interface with real-time processing and results display.

# 5. Advantages & Disadvantages

## 5.1 Advantages

- **Dual-surface architecture serves two distinct workflows from a single shared engine** — developers who want local git workflow integration use the CLI pre-commit hook with diff-aware analysis; teams who want to audit any public repository without local setup use the web platform. The shared @athena/core package guarantees identical analysis results across both surfaces, eliminating implementation drift.
- **Context-aware detection through repository fingerprinting** — unlike generic tools that apply the same thresholds to all code, Athena computes a baseline from the first scan and detects deviation from the project's own coding style. A codebase with consistently verbose comments won't false-flag human-written code that happens to be well-documented.
- **Hallucination detection fills a gap no existing tool addresses** — LLMs confidently generate imports for packages that don't exist and method calls for functions that aren't defined. Athena's hallucination detector validates imports against node_modules and checks named exports and method signatures, catching runtime failures that static linters cannot detect.
- **Fully offline CLI operation with zero external API costs** — the entire CLI analysis pipeline runs locally with no network calls, no rate limits, and no data transmitted externally. Security-conscious organizations can use the CLI in air-gapped environments without risking codebase exposure to third-party services.
- **Distributable as a real npm package with genuine adoption potential** — publishing to npmjs.com as @arsh342/athena means any developer can install the tool with a single command. The benchmark module provides quantifiable precision/recall/F1 metrics, giving the project academic rigor that positions it beyond a demonstration artifact.

## 5.2 Disadvantages

- **AI detection is heuristic-based, not deterministic** — the 11-signal scoring system produces probabilistic classifications, not ground-truth identification of AI-generated code. False positives (human code matching AI patterns) and false negatives (AI code using unusual patterns) are inherent limitations. The tool explicitly frames findings as elevated-risk sections for human review rather than absolute claims about code origin.
- **Requires Semgrep binary as a documented external dependency** — the CLI tool operates in graceful degradation mode (ESLint-only) if Semgrep is not installed, but the full security analysis capability requires Semgrep to be available in the system PATH. This creates an onboarding friction point that must be clearly documented.
- **Web platform sandbox is vulnerable to adversarial repositories** — while the 100MB size limit and 120-second timeout mitigate most resource exhaustion attempts, a specially crafted repository with many small files could still stress the AST parser. The 3-concurrent-scan cap and 10-scans/hour rate limit reduce but do not eliminate server resource risk under adversarial conditions.
- **Version 1 supports JavaScript and TypeScript only** — Python, Java, Go, and Rust codebases — which represent a significant portion of AI-assisted development activity — are entirely out of scope for v1. Teams working with polyglot repositories will receive analysis only on their JS/TS files, potentially missing vulnerabilities in other language sections.

# 6. Conclusion & Future Improvements

## 6.1 Conclusion

Athena addresses a timely and previously unsolved problem in the developer security tooling ecosystem. As AI coding assistants become standard components of software development workflows, the risk of unreviewed AI-generated code entering production codebases increases proportionally — yet no existing static analysis tool specifically targets this threat model. Athena fills this gap by introducing a risk prioritization layer that elevates scrutiny on code sections most likely to be unreviewed AI output, rather than treating all code as equally trustworthy.

The dual-surface architecture — CLI pre-commit gate with diff-aware analysis and web sandbox terminal for repository auditing — serves two distinct use cases from a single shared core engine. The monorepo structure with npm workspaces demonstrates real-world software engineering practices: package-level separation of concerns, shared type definitions, coordinated builds via tsup and Vite, and three independently testable packages under one repository.

The project satisfies all full-stack engineering requirements through its React frontend, Node.js/Express backend, WebSocket streaming architecture, and PostgreSQL persistence via Prisma ORM, while simultaneously delivering a publishable CLI artifact with a genuine use case that extends beyond academic submission. The benchmark module provides quantifiable precision, recall, and F1 metrics that give the detection engine academic credibility alongside its practical utility.

## 6.2 Future Improvements

- **LLM explanation layer —** optional Claude or Gemini API integration that generates plain-English explanations for why a flagged section is high-risk, transforming the severity report from a finding list into an actionable code review (opt-in, requires user's own API key).
- **Python language support —** Bandit static analysis integration and Python AST parsing via the ast module, extending Athena's detection to Python codebases which represent the majority of AI-assisted development in data science and ML contexts.
- **VS Code extension —** inline highlighting of AI-flagged sections directly in the editor with severity decorations, wrapping the same @athena/core engine in a VS Code extension API for real-time IDE feedback without leaving the editor.
- **GitHub App integration —** a GitHub App that automatically runs Athena on pull requests and posts inline review comments with severity classifications, bringing the pre-commit security gate into the PR review workflow for teams using GitHub.
- **ML-based classifier as v2 detection layer —** replacing or augmenting the heuristic scorer with a model trained on a labeled corpus of AI-generated vs human-written code, using the benchmark module's precision/recall framework to measure improvement over the heuristic baseline.
- **Multi-language support via tree-sitter —** tree-sitter provides a language-agnostic AST parsing layer for Go, Rust, Java, and C++, enabling Athena's core engine to operate across all major languages without language-specific parser implementations.

## 6.3 Potential Bottlenecks

**Technical Bottlenecks:**

- Semgrep subprocess latency — spawning a Semgrep process per flagged unit adds significant overhead; batching all flagged units into a single Semgrep invocation is the required optimization to meet the 10-second CLI scan target.
- AST parse performance on large files — TypeScript Compiler API is comprehensive but not optimized for speed; files exceeding the 100KB maxFileSize config limit are skipped to maintain performance targets.
- WebSocket memory accumulation — large repositories generating many log messages can accumulate significant memory in the WebSocket buffer if the client is slow to consume; backpressure handling must be implemented in the streaming service.

**Operational Bottlenecks:**

- Concurrent scan management — three simultaneous git clones and child processes each consuming up to 256MB of memory can approach Railway or Render free tier memory limits; a job queue (BullMQ) is required for production scaling.
- Heuristic calibration drift — as LLMs evolve their output style, the 11 signals may become less discriminative; the configurable weight system and benchmark runner enable signal recalibration without rebuilding the core engine.

**Regulatory Bottlenecks:**

- Repository privacy — the web platform clones public repositories only; private repository support would require GitHub OAuth and token handling with strict data retention policies to comply with repository access terms.
- Hosting costs — Railway and Render free tiers have compute and memory limits; sustained usage with concurrent scans will require paid tiers, adding recurring operational expenses.

# 7. References


1. Harness Inc., "State of Software Delivery 2025." Harness Engineering Report, 2025. [Online]. Available: https://harness.io
2. OWASP Foundation, "OWASP Top Ten." Open Web Application Security Project, 2021. [Online]. Available: https://owasp.org/www-project-top-ten/
3. Semgrep Inc., "Semgrep Documentation and Rule Registry." 2024. [Online]. Available: https://semgrep.dev/docs
4. Microsoft Corporation, "TypeScript Compiler API." TypeScript GitHub Wiki, 2024. [Online]. Available: https://github.com/microsoft/TypeScript/wiki/Using-the-Compiler-API
5. Yelp Engineering, "detect-secrets: Enterprise friendly secret detection." GitHub, 2024. [Online]. Available: https://github.com/Yelp/detect-secrets
6. ESLint Community, "eslint-plugin-security." npm Registry, 2024. [Online]. Available: https://github.com/eslint-community/eslint-plugin-security
7. xterm.js contributors, "xterm.js: A terminal for the web." GitHub, 2024. [Online]. Available: https://github.com/xtermjs/xterm.js
8. Prisma, "Prisma ORM Documentation." 2024. [Online]. Available: https://www.prisma.io/docs
9. Vite, "Vite Build Tool Documentation." 2024. [Online]. Available: https://vitejs.dev
10. T. J. Mahr, "Stylometric Analysis of Source Code for Authorship Attribution." ACM SIGPLAN Notices, 2019. [Online]. Available: https://dl.acm.org/doi/10.1145/3357390.3361037